ChatGPT said:

The Storytelling Mirror — project blueprint

The goal is to transform your existing React camera web app into a playful, antique-inspired cultural guide for the Peranakan expo. Visitors should feel as though they are interacting with an ornate Chinese mirror that reveals the stories behind exhibits, while the experience remains simple enough for anyone to use on their phone.

We'll keep your chosen **ornate mirror with hanging tassels** as the central design element and build around four screens:



1. Welcome screen

Core screen

The first impression. Introduces the experience with the mirror illustration, a short welcome message, and a prominent **Start Exploring** button.

The visitor should understand what to do immediately.



2. Camera tour screen

Core screen

Opens the rear camera, frames the QR scanning area with a decorative mirror-inspired interface, and guides the visitor to scan exhibits as they explore the expo.



3. Exhibit information screen

Core screen

When a QR code is recognized, the app reveals the exhibit's name, photo, history, cultural significance, and related imagery in a readable, heritage-inspired layout.



4. Camera collage screen

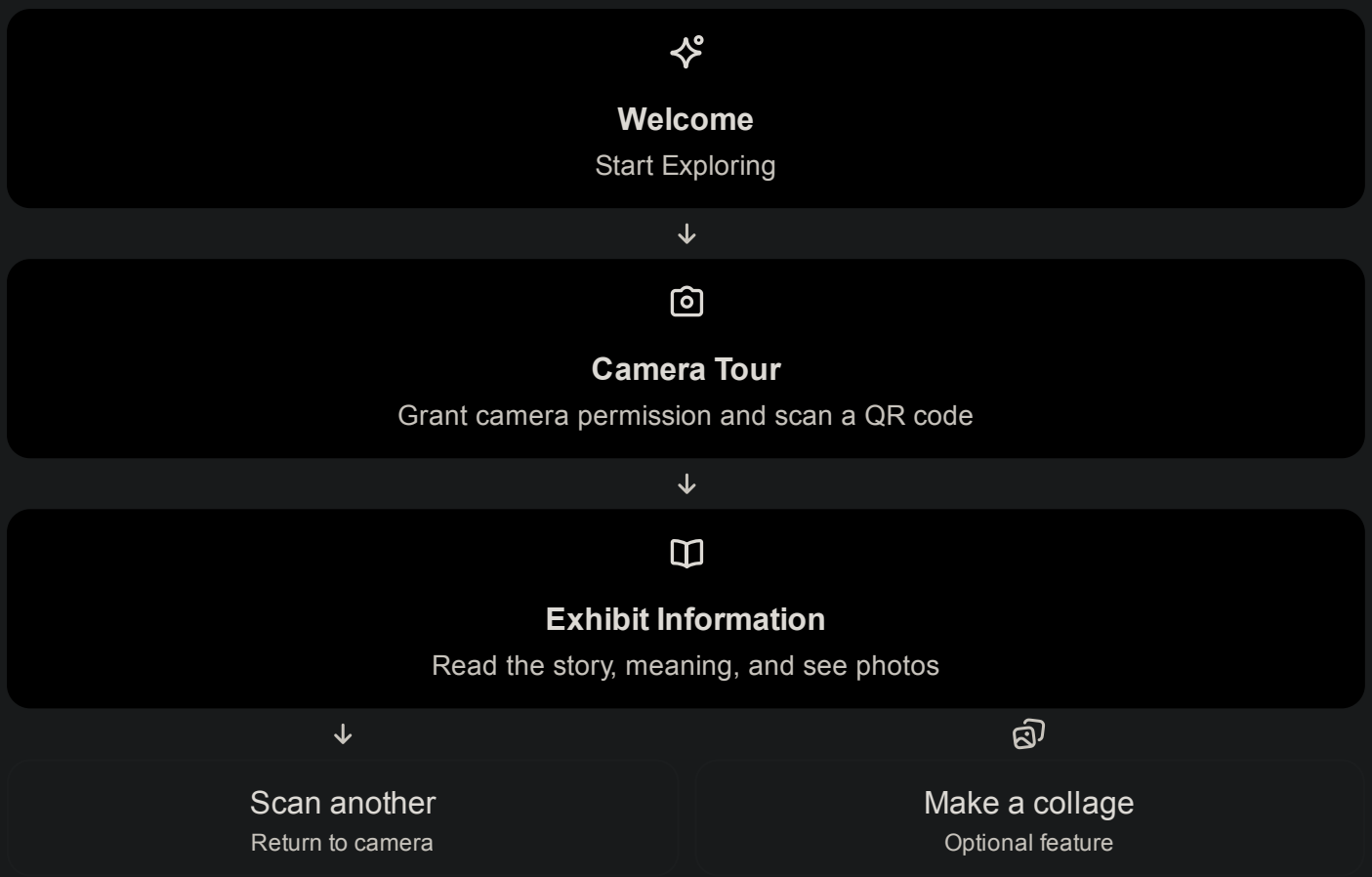
Optional feature

Gives visitors a way to capture or collect exhibit photos and arrange them into a personal cultural-expo collage they can save or share.

One important distinction: the first three screens form the minimum usable experience. The collage should be added after scanning and exhibit information work reliably.

1. Plan the visitor journey

The main flow should be short and predictable.



The main navigation decision is to make **Scan another exhibit** the primary next action from the information screen. Visitors should not have to return to the home screen every time they scan a new item.

2. Define the requirements for each screen

Screen 1 — Welcome



Visual direction

- Deep vermilion background or warm ivory paper.
- Large ornate mirror with floral inlay and a visible hanging tassel.
- Serif heading: *The Storytelling Mirror*.
- Small decorative flourishes, not cluttered illustrations.
- A large, clear Start Exploring button.

Required behavior

- Open the camera tour on tap.
- Explain why camera access is needed before requesting permission.

- Provide a short instruction: “Scan the QR code beside any exhibit to uncover its story.”

Screen 2 — Camera tour



Visual direction

Keep the live camera feed as the main element. Use the ornate mirror frame as a border around the scanning area, with tassels hanging from the frame edges rather than covering the QR code.

Required behavior

- Request access to the rear camera when scanning begins.
- Detect a QR code and prevent repeated processing of the same scan.
- Show a short scan confirmation or transition animation.
- Provide back, retry, and optionally flashlight controls.
- Handle camera denial, missing cameras, and invalid QR codes gracefully.

The scanning target must remain large and unobstructed, even on narrow phones.

Screen 3 — Exhibit information



Visual direction

Present the information like a page from a treasured heritage journal. Use ornamental separators, framed exhibit photographs, muted cream panels, and red or gold accents.

Required behavior

- Show exhibit name, image, and a short introduction.
- Display the full story and cultural background.
- Explain symbolism or traditional meaning when relevant.
- Offer a gallery of related images.
- Include **Scan Next Exhibit** and **Add to Collage** actions.

Use tabs for Story, Symbolism, and Gallery to avoid making the user scroll through one enormous page.

Screen 4 — Optional camera collage



Visual direction

Imagine a digital keepsake album: exhibit photographs arranged on a parchment background, decorated with subtle floral corners, stamps, and tassel details.

Suggested behavior

- Let visitors capture a new photo or select an eligible image.
- Add images from previously scanned exhibits.
- Arrange, remove, or reorder photographs.
- Choose a simple collage layout.
- Save the finished collage as an image or share it through the phone's supported sharing interface.

Keep this screen optional and ensure that photos stay on the visitor's device unless they explicitly choose to share them.

One product decision worth making now: should the collage use **only photos of the exhibits**, or should visitors also be able to take selfies alongside their discoveries? I'd build the first version around exhibit photos and add selfie capture only if it's part of the expo experience.

3. Establish the visual design system

The moodboard should translate into a small, consistent set of reusable styles instead of independently styling every screen.

Color palette

Vermilion	Rosewood	Antique gold
#B32222	#4A2E26	#D4AF37
Copy hex	Copy hex	Copy hex

Warm ivory	Peranakan teal	Blush pink
#F7EAD9	#2C9C8D	#E7A7B3
Copy hex	Copy hex	Copy hex

Materials and decorative language



Carved rosewood

Use it for the main frame, with subtle shadows to evoke depth.



Antique gold

Reserve it for the mirror edge, fine dividers, decorative details, and selected icons.



Red silk tassels

Make these the signature visual feature. Use small animations only for transitions or successful scans.



Peranakan floral ornamentation

Add these to corners, section dividers, and the collage frame. Keep the QR scanning area relatively plain.

For typography, use an elegant serif for titles and a clear sans-serif for instructions and body text. Apply a consistent minimum touch target of approximately 44 × 44 CSS pixels, use adequate text contrast, and respect the phone's safe areas.

4. Plan the React implementation

We should preserve the existing project's dependencies and camera implementation wherever possible. There is no reason to migrate frameworks or rewrite the scanner before inspecting the source.

A proposed component structure is:

Plain text

```
src/
  components/
    OrnateMirrorFrame.jsx
    DecorativeButton.jsx
    CameraViewport.jsx
    ExhibitCard.jsx
    ContentTabs.jsx
    CollageGrid.jsx
  pages/
    WelcomePage.jsx
    CameraTourPage.jsx
    ExhibitInfoPage.jsx
    CollagePage.jsx
  data/
    exhibits.js
  services/
    qrScanner.js
    exhibitService.js
  styles/
```

This is a suggested structure, not a requirement to move all your existing files. If the project already uses different component names or a routing convention, we'll adapt to it.

How the screens connect to the data

Each scanned QR code should identify a specific exhibit, and the app should retrieve the corresponding curated information.

An example exhibit record:

JavaScript

```
{
  id: "EXPO-001",
  title: "Peranakan Beaded Slippers",
  period: "19th Century",
  summary: "An introduction to the exhibit.",
  story: "The full cultural and historical story.",
  symbolism: "The meaning of its decorative elements.",
  images: [
    "/images/beaded-slippers-1.jpg",
    "/images/beaded-slippers-2.jpg"
  ]
}
```

The values above are illustrative; actual exhibit details need to come from your expo's content.

For the first prototype, this information can live in a local JavaScript data file. If the expo already has an API or database, we'll connect to that instead.

The QR scanner must validate recognized codes against the expo's expected identifiers. It should not automatically navigate to arbitrary URLs found in scanned codes.

5. Build in milestones

Milestone 1 — Audit the existing app

Inspect its source files, identify the current camera library, determine how QR scanning works, and understand the current navigation and styling.

Deliverable: an integration plan that preserves working code.

Milestone 2 — Implement the design system and welcome screen

Create color and typography tokens, the mirror frame, tassel details, decorative buttons, and responsive layout.

Deliverable: a consistent visual style that works on mobile.

Milestone 3 — Complete the camera tour

Preserve or adapt the scanner, build the camera frame, add clear camera-permission handling, and ensure repeated scans do not trigger duplicate results.

Deliverable: reliable scanning on target mobile browsers.

Milestone 4 — Build the information screen

Connect scan results to exhibit data, add Story, Symbolism, and Gallery sections, and implement the next-exhibit flow.

Deliverable: a complete scanning-to-discovery experience.

Milestone 5 — Add the optional collage

Implement photo capture or selection, exhibit collection, layout editing, image export, and supported sharing.

Deliverable: a visitor keepsake feature.

Milestone 6 — Test and deploy

Test physical QR codes, weak network connections, denied camera permissions, small screens, long descriptions, and the hosted HTTPS version.

Deliverable: an expo-ready web app.

6. Definition of done

Before calling the app finished, we should verify that:

- Visitors can start scanning without creating an account or installing an app.
- The rear camera opens reliably on supported phones over HTTPS.
- A valid QR code brings up the correct exhibit every time.
- Invalid codes and unavailable exhibit records produce helpful messages instead of a broken screen.
- Visitors can read information, browse images, and scan the next exhibit without restarting.
- The ornamental design never blocks camera input, controls, or important information.
- The optional collage can be saved successfully on supported devices, with clear feedback if a browser does not support a particular sharing feature.

7. What we need from your current project

I checked your existing camera app ↗, but the page inspection didn't expose its rendered interface or source code. So I haven't yet verified which scanning library, routes, or styling system it uses.

The best next step is to **upload a ZIP of your current React project's source code**, excluding `node_modules`, build output, and any `.env` files containing secrets. Include `package.json`, the `src` directory, and the relevant public assets.

That lets us identify exactly what to preserve, what to modify, and what to add—before making any changes to your working scanner.